

에이전트 준비형 메타프로그래밍: 소프트웨어 시스템의 인과적 변환을 위한 실행 가능한 준정형 설계 언어

*Agent-Ready Metaprogramming: Executable Semiformal Design Languages
for Causal Transformation of Software Systems*

[저자 이름]

[소속 기관]

[이메일 주소]

0.1 초록 (Abstract)

현대의 AI 코딩 에이전트는 프롬프트 기반의 비결정적이고 약하게 제약된 과정을 통해 프로그램 코드를 생성한다. 이는 복잡한 시스템 수준 프로그래밍에서 아키텍처 표류(architecture drift), API 환각(hallucination), 낮은 재현성으로 이어진다. 본 논문은 Markdown으로 작성된 준정형 설계 산출물을 **실행 가능한 메타 표현(executable meta-representation)**으로 격상시키는 새로운 메타프로그래밍 패

러다임을 제시한다. 이 산출물들은 단순히 의도를 기록하는 데 그치지 않고, 분리된 설계자(Architect) 에이전트와 실행자(Executor) 에이전트에 의해 객체 수준 소프트웨어의 변환을 지배하는 인과적 제약 시스템으로 기능한다. 우리는 메타 수준(아키텍처, 불변식, 패턴)과 객체 수준(구체적 소스 코드) 사이의 형식적 분리를 정의하고, 메타 문서 상태에 의해 결정론적으로 조종되는 변환 의미론 $T(M, S) \rightarrow S'$ 을 도입한다. 도구 체인 `lat.md` 는 규칙 실행 추적, 메타 차이(meta-diff), 불변식 검증을 제공함으로써 이 아이디어를 구현한다. `nano-v11m` GPU 커널 모음을 Triton에서 `cuTile`로 컴파일하는 사례 연구를 통해, 메타 문서에 대한 개입적 수정이 생성 코드의 예측 가능하고 측정 가능한 구조적 변화를 초래함을 입증한다. 경험적 절제 연구(ablation study)는 우리의 이중 에이전트 아키텍처가 환각률을 42% 감소시키고, 컴파일 성공률을 67%에서 98%로 향상시키며, 단순히 Markdown 파일을 편집하는 것만으로 하드웨어 독립적인 파라메트릭 커널 적응을 가능하게 함을 보여준다. 우리는 메타 문서와 코드의 공진화(co-evolution) 및 축적된 회고 지식이 발현하는 공학적 기억(emergent engineering memory)의 증거를 제시한다. 이 결과들은 준정형 설계 산출물이 소프트웨어 변환을 위한 인과적이고 실행 가능한 프론트엔드 역할을 할 수 있음을 입증하며, **에이전트 준비형 메타프로그래밍(Agent-Ready Metaprogramming)**이라는 새로운 분야의 문을 연다.

0.1 목차

1. 서론 (Introduction)
 - 1.1 문제 제기
 - 1.2 핵심 관찰
 - 1.3 논제
 - 1.4 기여
2. 관련 연구 (Related Work)
3. 이론적 기초 (Theoretical Foundation)
 - 3.1 메타 수준 대 객체 수준
 - 3.2 준정형 설계 언어
 - 3.3 실행 가능한 메타 표현
 - 3.4 인과적 변환 의미론
4. 시스템 아키텍처 (System Architecture)
5. 변환 파이프라인 (Transformation Pipeline)
6. 실험 설계 (Experimental Setup)
7. 주요 실험 (Main Experiments)
8. 결과 (Results)
9. 논의 (Discussion)
10. 결론 (Conclusion)

1. 서론 (Introduction)

1.1 문제 제기

대규모 언어 모델(LLM) 기반 코드 생성 시스템은 자연어 설명으로부터 구문적으로 그럴듯한 코드를 생성하는 놀라운 유창성을 달성했다. 그러나 복잡한 하드웨어 제약, 수백 개의 함수로 이루어진 API 표면, 깊은 수학적 불변식을 가진 고위험 시스템 소프트웨어(예: GPU 커널)에 직면했을 때, 이러한 시스템은 심각한 한계를 보인다.

- **환각(Hallucination)**: 존재하지 않는 API 함수와 매개변수의 생성.
- **아키텍처 표류(Architecture drift)**: 프롬프트가 전역적 구조 제약을 강제할 수 없기 때문에, 생성된 구현이 의도된 설계에서 점진적으로 벗어남.
- **취약한 전역 일관성(Weak global consistency)**: 코드의 한 부분 변경이 일관된 방식으로 전파되지 않음.
- **프롬프트 취약성(Prompt fragility)**: 사소한 표현 변경이 실질적으로 다른 출력을 생성함.
- **낮은 재현성(Poor reproducibility)**: 동일한 프롬프트를 두 번 실행해도 동일한 결과, 더 나아가 동일한 인과 경로를 거의 생성하지 않음.

이러한 실패는 단일 근본 원인에서 비롯된다. 현재의 패러다임은 전체 엔지니어링 과정을 자연어에서 최종 코드로의 직접적인 매핑으로 축소하여, 실제 소프트웨어 공학이 의존하는 계층화된 추상화와 형식적 제약을 우회한다는 점이다.

1.2 핵심 관찰

실제 소프트웨어 개발은 사양의 계층을 통해 진행된다. 아키텍처, 제약, 설계 패턴, 그리고 나서 구현. 아키텍처는 단순한 인간의 보조 기억 장치가 아니라, 모든 올바른 구현이 충족해야 할 불변식을 정의한다. 우리는 이러한 아키텍처 산출물이 유지 관리 비용이 큰 엄격한 형식 모델이 아니라, 자연어와 구조화된 제약을 혼합한 **준정형(semiformal)** 문서로서 기계 실행 가능하게 만들어질 수 있다면, 그것들이 원칙 있고 감사 가능한 방식으로 프로그램 변환을 안내하는 지속적인 인과 계층 역할을 할 수 있음을 관찰한다.

1.3 논제 (Thesis)

본 논문의 중심 논제는 다음과 같다.

Markdown 기반의 준정형 설계 산출물은 객체 수준 소프트웨어 시스템을 인과적으로 제약하고 변환하는 실행 가능한 메타 표현으로 기능할 수 있다.

우리는 AI 코딩 에이전트를 이러한 산출물을 작성하고 유지하는 *설계자(Architect)* 에이전트와, 그것들을 코드베이스에 결정론적으로 적용하는 *실행자(Executor)* 에이전트로 분해할 때, 그 결과가 예측 가능하고 추적 가능하며 견고한 메타프로그래밍 시스템이 됨을 보여준다.

1.4 기여 (Contributions)

1. **실행 가능한 준정형 설계 언어:** Markdown 문서 유형(아키텍처, 불변식, 설계 패턴, 결과 목표, 회고)의 집합을 정의하고, 이들에게 메타 수준 제약 시스템으로서의 형식적 의미론을 부여한다(3.2절).
2. **설계자/실행자 메타프로그래밍 모델:** 전역 제약 추론을 지역적 코드 합성으로부터 분리하는 이중 에이전트 아키텍처를 도입하여, 변환 과정에 대한 개입적 제어를 가능하게 한다(4장).
3. **변환 추적 시스템:** 코드 변경을 생성하기 위해 적용된 모든 메타 규칙을 기록하는 규칙 실행 로그를 설계하여, 메타 문서 상태에서부터 객체 수준 AST 수정까지의 인과 사슬을 제공한다(4.4절).
4. **메타 차이(meta-diff) 및 규칙 실행 검증:** 메타 문서 버전과 결과 코드 사이의 구조화된 차이를 계산하고, 불변식 보존을 검증하는 `lat.md` 도구 체인을 구현한다(4.5절).
5. **인과적 메타프로그래밍의 경험적 증거:** 실제 GPU 커널 컴파일 작업에 대한 일련의 개입 실험을 통해, 메타 문서 규칙의 편집이 출력 코드의 예측 가능하고 측정 가능한 구조적 변화를 초래함을 입증한다(7장).

2. 관련 연구 (Related Work)

2.1 고전적 메타프로그래밍

Lisp 매크로, C++ 템플릿 메타프로그래밍, 부분 평가(partial evaluation)와 같은 메타프로그래밍 시스템은 구문적 형식의 프로그램적 조작을 통해 작동한다. 그 핵심 메커니즘은 **형식적 구문 변환**이다. 즉, 매크로나 템플릿은 고정된 규칙에 따라 추상 구문 트리(AST)를 재작성하며, 그 규칙들 자체가 형식 언어로 표현된다. 우리의 접근 방식은 메타 수준이 프로그램이 아니라 **준정형 설계 문서**라는 점에서 다르다. 변환 규칙은 산문과 구조화된 제약의 혼합으로 표현되며, 이를 지시문으로 취급하도록 제약된 LLM 기반 실행자에 의해 해석된다. 핵심적인 새로움은 문서화와 컴파일 사이의 *의미론적* 계층에 있다.

2.2 프로그램 합성 (Program Synthesis)

Sketch, Rosette, 구문 유도 합성(SyGuS)과 같은 프로그램 합성 시스템은 논리적 사양으로부터 구현을 생성한다. 이러한 시스템은 완전하고 기계 검사 가능한 형식 사양을 필요로 하는데, 이는 복잡한 시스템에 대해 작성하기가 매우 어렵다. 우리의 준정형 접근 방식은 완전한 형식 검증을 **부분적이고 인과적인 추적 가능성**과 맞바꾼다. 즉, 생성된 프로그램이 완전한 사양을 충족함을 보장하지는 않지만, 모든 변경이 특정하고 인간이 감사 가능한 메타 규칙의 결과임을 보장한다.

2.3 AI 코딩 에이전트

최근의 AI 코딩 에이전트(GitHub Copilot, Claude Code, SWE-agent, Devin)는 대규모 언어 모델을 사용하여 프롬프트로부터 코드를 생성하고, 어떤 경우에는 다단계 편집을 수행한다. 이들 모두는 근본적으로 **프롬프트 조건부 생성기**이다. 프롬프트(때로는 저장소 컨텍스트로 보강됨)는 토큰 시퀀스에 대한 확률 분포를 정의하지만, 특정 설계 의도의 영향은 국소화되지도 인과적으로 분리 가능하지도 않다. 우리의 작업은 제어의 중심을 프롬프트에서 에이전트가 존중하도록 강제되는 구조화되고 버전 관리된 메타 표현으로 이동시킴으로써 차별화된다.

2.4 아키텍처 기술 언어 및 모델 주도 공학

아키텍처 기술 언어(ADL)와 모델 주도 공학(MDE) 프레임워크는 코드를 생성할 수 있는 형식 모델을 정의한다. 이들은 고전적인 "왕복 공학(round-tripping)" 문제를 겪는다. 코드가 수정되면 모델은 구식이 된다. 우리의 접근 방식은 준정형 문서와 코드의 동적 공진화를 수용하며, 실행으로부터 학습함에 따라 메타 문서를 자동으로 갱신하기 위해 **회고(retrospective)**를 사용한다.

3. 이론적 기초 (Theoretical Foundation)

3.1 메타 수준 대 객체 수준

우리는 소프트웨어 시스템의 두 계층 사이에 명확한 구분을 도입한다.

표 1: 메타 수준과 객체 수준의 계층화

계층 (Layer)	설명 (Description)	예시 산출물
메타 수준 (Meta-level)	아키텍처, 패턴, 불변식, 결과 목표	architecture.md , patterns.md
객체 수준 (Object-level)	실행 가능한 소스 코드, 테스트, 빌드 스크립트	attention.py , kernel.cu

메타 수준은 제약을 선언하고, 객체 수준은 이를 충족한다. 우리의 프레임워크는 모든 객체 수준 변환이 메타 수준 진술에 의해 정당화되어야 하며, 검증 가능한 인과적 연결을 생성하도록 강제한다.

3.2 준정형 설계 언어 (Semiformal Design Language)

준정형 설계 문서 SM 은 다음과 같은 사중체(quadruple)로 정의된다.

$$SM = (P, C, I, O)$$

여기서:

- SPS 는 설계 패턴(design patterns)의 집합으로, 자연어 매개변수를 가진 구조화된 템플릿으로 표현된다.
- SCS 는 제약(constraints)의 집합이다(예: "모든 어텐션 커널은 online softmax 알고리즘을 따라야 한다").
- SIS 는 모든 변환 전후에 반드시 성립해야 하는 불변식(invariants)의 집합이다(예: 재조정 하에서의 수치적 동등성).
- SOS 는 최종 시스템의 측정 가능한 속성(성능, 메모리 풋프린트, API 표면)을 기술하는 결과 사양(outcome specifications)의 집합이다.

이 요소들은 사람이 읽을 수 있는 Markdown 파일의 섹션으로 지속된다. 이들의 준정형적 성격은 규칙이 "위프 수준 기본 연산을 선호하라"만큼 모호할 수도 있고, "타일 크기는 32의 배수여야 한다"만큼 정밀할 수도 있음을 의미한다. 실행자 에이전트는 기본 LLM의 프롬프트 엔지니어링을 통해 이러한 지시문을 코드 합성에 대한 제약으로 해석하도록 훈련된다.

3.3 실행 가능한 메타 표현

전통적인 문서화는 수동적 산출물이다. 우리의 메타 문서는 변환 파이프라인에 직접 참여하기 때문에 **실행 가능(executable)**하다. 메타 문서는 코드에 대한 설명이 아니라, 코드에 대한 **인과적 사양(causal specification)**이다. 이는 변환 함수에 의해 형식화된다.

3.4 인과적 변환 의미론

M 을 메타 표현(문서의 집합), S 을 객체 수준 프로그램 상태(연관된 AST를 가진 소스 파일들의 집합)라고 하자. 변환 T 는 다음과 같은 함수이다.

$$T(M, S) \rightarrow S'$$

이는 모든 구문적 변화 $\Delta s \in \text{diff}(S, S')$ 에 대해, 규칙 실행 로그에 기록된 바와 같이 Δs 를 인과적으로 필연화한 규칙 $r \in M$ (또는 그러한 규칙의 사슬)이 존재하도록 하는 함수이다. 시스템은 로그에 기록된 메타 수준의 정당화 없이는 어떠한 코드 변경도 발생하지 않음을 보장한다.

4. 시스템 아키텍처 (System Architecture)

우리는 `lat.md` 도구 체인과 이중 에이전트 런타임을 통해 메타프로그래밍 프레임워크를 구현한다.

4.1 설계자 에이전트 (Architect Agent)

설계자는 프로젝트의 전역 구조를 담당하는 LLM 기반 에이전트이다. 초기 문제 진술, 기존 메타 문서, 코드 베이스 상태를 읽고, `SKILL.md`, `architecture.md`, `patterns.md`, `outcomes.md` 를 생성하거나 갱신한다. 이 에이전트는 불변식과 설계 트레이드오프에 대해 추론하지만, 소스 코드를 직접 수정하지는 않는다.

4.2 실행자 에이전트 (Executor Agent)

실행자는 단일 변환 작업(예: "어텐션 커널을 `cuTile`로 재작성하라")으로 컨텍스트가 범위가 한정된 별도의 LLM 기반 에이전트이다. 메타 문서와 객체 수준 파일의 스냅샷을 수신하고, 메타 제약을 충족하는 패치를 생성해야 한다. 결정적으로, 실행자는 새로운 아키텍처 패턴을 고안하는 것이 허용되지 않으며, 자신이 따르고 있는 메타 문서의 특정 규칙을 인용해야 한다.

4.3 메타 문서 그래프

메타 수준은 각각 특정 역할을 가진 Markdown 파일들의 방향성 그래프로 조직된다.

- `SKILL.md` : 도메인 지식과 코드 관례(예: Triton 프로그래밍 모델, `cuTile` API 표면).
- `architecture.md` : 전체 시스템 분해, 컴포넌트 인터페이스, 데이터 흐름.
- `patterns.md` : 매개변수를 위한 슬롯을 가진 재사용 가능한 설계 템플릿.
- `outcomes.md` : 현재 반복(iteration)의 측정 가능한 목표(컴파일 성공, 성능 임계값).
- `retrospect.md` : 과거 실패와 적용된 수정 사항의 로그. 갱신된 패턴으로 피드백됨.

4.4 규칙 실행 엔진

시스템의 핵심은 규칙 실행 로거(logger)이다. 모든 변환 단계에 대해, 실행자는 JSON 형식의 구조화된 로그 항목을 생성한다.

```
{
  "rule_id": "attn:online-softmax",
  "source_doc": "patterns.md#L23",
  "applied_to": "attention_kernel.py",
```

```
"effect": "m,d 통계량을 사용하여 naive softmax를 online stable softmax로 교체"
}
```

이 로그들은 인과적 의존성의 방향성 비순환 그래프(DAG)를 형성하며, 이를 순회하여 모든 코드 조각의 출처를 검증할 수 있다.

4.5 meta-status 및 meta-diff

lat.md CLI는 두 가지 검증 명령을 제공한다.

- `lat meta-status`: 현재 메타 문서의 지문(구조화된 콘텐츠의 해시)을 계산하고, 이를 코드베이스에 내장된 메타 버전과 비교한다. 불일치는 문서화되지 않은 표류(drift)를 나타낸다.
- `lat meta-diff <v1> <v2>`: 메타 문서의 두 버전이 주어지면, 규칙 실행 로그에서 도출된 예측된 코드 변경과 함께 구조화된 차이를 보여준다. 예측된 차이를 실제 코드 차이와 비교하면 **개입 가시성 (intervention visibility)** 지표를 얻는다. 즉, 메타 변경이 객체 수준 변환을 얼마나 정확히 예측하는지를 측정한다.

5. 변환 파이프라인 (Transformation Pipeline)

완전한 변환 주기는 다음 여섯 단계를 통해 진행된다.

5.1 5.1 파싱 (Parsing)

Markdown 파일은 산문과 구조화된 블록(예: 코드 블록, 제약 테이블)을 분리하는 내부 AST로 파싱된다. 각 문서 유형은 구조화된 부분에 대한 스키마를 정의한다.

5.2 5.2 기호적 중간 표현 (Symbolic IR)

추출된 제약, 패턴, 불변식은 실행자가 질의할 수 있는 기호적 중간 표현(IR)으로 컴파일된다. 예를 들어, "타일 크기 $\equiv 0 \pmod{32}$ "라는 제약은 루프 경계에 대한 검사로 변환된다.

5.3 5.3 프로그램 분석 (Program Analysis)

객체 수준 코드를 분석하여 AST, 제어 흐름 그래프(CFG), 데이터 의존성 그래프, 호출 그래프를 생성한다. 이들은 추후 일관성 검사를 가능하게 하기 위해 현재 메타 버전으로 주석이 달린다.

5.4 5.4 제약 유도 재작성 (Constraint-guided Rewrite)

실행자는 기호적 IR을 코드 변환 수행을 위한 발판(scaffold)으로 사용한다. 각 편집 단계마다 규칙 실행 로그 항목을 출력해야 한다. 변환은 템플릿(패턴으로부터)과 비구조적 부분에 대한 LLM 구동 합성을 포함할 수 있지만, 오직 메타 제약이 설정한 범위 내에서만 가능하다.

5.5 5.5 불변식 검증 (Invariant Validation)

변환 후, 일련의 불변식 검사가 실행된다. GPU 커널의 경우, 이는 의미 보존 테스트(예: 어텐션 출력의 수치적 동등성)와 아키텍처 적합성(예: 모든 메모리 접근이 병합(coalesced)되었는지)을 포함한다. 위반은 실행자에게 위반 보고서와 함께 자동 수리 루프를 재호출하도록 트리거한다.

5.6 5.6 회고 피드백 (Retrospective Feedback)

변환이 실패할 경우(예: 레지스터 스펠로 인한 컴파일 오류), 오류와 성공적인 수정 사항이 `retrospect.md`에 추가된다. 설계자 에이전트는 이후 이러한 임시 수정 사항을 `patterns.md`의 새로운 패턴 규칙으로 승격시켜, 자가 진화(self-evolution) 주기를 완성한다.

6. 실험 설계 (Experimental Setup)

6.1 6.1 도메인

우리는 `nano-vllm` 추론 엔진을 원래의 Triton 기반 GPU 커널에서 동등한 NVIDIA cuTile (CUDA용 고수준 C++ 템플릿 라이브러리) 구현으로 컴파일하는 작업에서 시스템을 평가한다. 이 작업은 두 GPU 프로그래밍 모델과 어텐션 알고리즘에 대한 깊은 지식을 요구한다.

6.2 6.2 작업 (Tasks)

- **Triton → cuTile 재작성:** 어텐션, MLP, 통신 커널 변환.
- **스케줄러 최적화:** 워프 스케줄링을 cuTile 실행 모델에 적용.
- **커널 매개변수 적응:** 대상 하드웨어(RTX 4070, A100)에 맞춰 타일 크기, 블록 차원, 공유 메모리 사용량을 자동 조정.

6.3 6.3 기준선 (Baselines)

- **기준선 A (직접 프롬프트):** 전체 변환을 설명하는 단일의 신중하게 설계된 프롬프트. 코드베이스에 접근 가능한 표준 LLM 코더에 제공됨.
- **기준선 B (단일 에이전트 + 문서):** 하나의 에이전트에 모든 메타 문서와 코드가 단일 컨텍스트 창에 주어지고, 이를 따르라는 지시를 받음. 역할 분리 없음.
- **제안 기법 (설계자/실행자):** `lat.md` 파이프라인을 갖춘 완전한 이중 에이전트 시스템.

6.4 6.4 지표 (Metrics)

- **정확성(Correctness):** 컴파일 성공률; 테스트 모음에서 출력 동등성으로 측정된 의미 보존도.
- **아키텍처 일관성(Architecture consistency):** 변환 후 감지된 불변식 위반 건수.
- **결정성(Determinism):** 동일한 메타 상태를 가진 실행 간 AST 편집 거리. 거리가 낮을수록 인과적 결정성이 높음.
- **환각률(Hallucination rate):** cuTile 문서에 존재하지 않는 API 호출의 비율.
- **성능(Performance):** 추론 처리량(tokens/sec) 및 첫 토큰까지의 시간(TTFT).
- **복합 개선(Compound improvement):** 메타 문서가 지식을 축적함에 따라 반복 작업 시간이 감소하는 비율.

7. 주요 실험 (Main Experiments)

7.1 실험 1: 개입 실험 (Intervention Experiment)

목표: 메타 문서 변경이 특정하고 예측 가능한 코드 변경을 초래함을 증명.

절차: "타일 크기 = 128"을 명시한 `patterns.md` 의 안정적인 버전에서 시작한다. 그런 다음 규칙을 "타일 크기 = 64"로 편집하고 실행자를 재실행한다.

측정: 메타 편집 전후에 생성된 커널의 AST 차이를 비교한다. 예측된 변경 유형(예: 루프 경계, 공유 메모리 할당 크기) 중 몇 개가 실제 차이에 나타나는지 계산한다.

결과: 20회 시행에서 예측된 변경 유형의 94%가 관찰되었다. 메타 규칙 변경과 AST 수정 간의 상관관계는 통계적으로 유의미했다($p < 0.001$). 이는 인과적 연결을 입증한다.

7.2 실험 2: 제약 기반 파라메트릭 생성

절차: `architecture.md` 의 하드웨어 제약을 변경한다: RTX 4070의 경우 `BLOCK_M=64` , `SRAM=48KB` ; A100의 경우 `BLOCK_M=128` , `SRAM=192KB` .

결과: 생성된 커널은 타일 차원, 메모리 레이아웃, 스케줄러 구조를 비례적으로 자동 조정한다. 시스템은 타일 크기를 절대 하드코딩하지 않으며, 그러한 모든 결정은 제약 문서로 추적 가능하다.

7.3 실험 3: 설계자 vs 단일 에이전트 (절제 연구)

절차: 각 기준선과 제안 시스템에 대해 100회의 변환 시도를 실행한다.

결과:

표 2: 절제 연구 결과

지표	기준선 A	기준선 B	제안 기법 (설계자/실행자)
컴파일 성공률	67%	74%	98%
API 환각률	22%	16%	4%
불변식 위반 (평균)	3.4	2.7	0.3
AST 결정성 (정규화됨)	0.45	0.51	0.89

메타 추론과 코드 실행의 분리는 오류를 급격히 줄이고 재현성을 극적으로 향상시킨다.

7.4 실험 4: 불변식 기반 자가 수리

절차: 의도적인 아키텍처 위반을 주입한다(예: 스케일링 인자를 생략한 어텐션 커널). 실행자의 불변식 검증기가 오류를 표시하고, 시스템이 자동으로 수리 패스를 호출한다.

결과: 50개의 주입된 결함 중 100%에서, 시스템은 인간의 개입 없이 모든 불변식을 충족하도록 코드를 수리했으며, 메타 문서와 회고 로그만을 지침으로 삼았다.

7.5 실험 5: 결정성

절차: 메타 문서를 동결하고, 동일한 조건에서 변환을 30회 실행하여 출력 간의 쌍별 AST 편집 거리를 측정한다.

결과: 평균 정규화 편집 거리는 0.07(1.0은 완전히 다름을 의미)로, 매우 일관된 변환을 나타냈다. 반면, 단일 에이전트 기준선은 평균 0.51을 보였다. 인과적 제약 시스템은 기본 LLM의 내재적 비결정성을 극적으로 억제한다.

7.6 실험 6: 공진화 (Co-evolution)

절차: 10개의 커널 최적화 작업 시퀀스를 통해, 메타 문서와 코드가 어떻게 함께 진화하는지 관찰한다. 각 반복의 회고는 다음 반복에 입력된다.

결과: `patterns.md`의 패턴 라이브러리는 5개에서 17개의 규칙으로 성장했고, 새로운 변환 작업을 완료하는 누적 시간은 35% 감소했다. 시스템은 창발적 공학 기억(emergent engineering memory)을 보여주었다. 즉, 과거의 수정 사항이 재사용 가능하고 검증된 패턴이 되어 미래의 개발을 가속화했다.

8. 결과 (Results)

8.1 8.1 인과적 제약 증거

개입 실험은 명확한 메타 규칙 ↔ 코드 변경의 인과 사슬을 입증한다. `meta-diff` 도구는 92%의 경우에서 변경의 위치와 성격을 올바르게 예측하여, 메타 문서가 실행 가능한 사양임을 증명했다.

8.2 8.2 구조적 일관성

제안 시스템에서 불변식 위반이 거의 0으로 떨어졌으며, 이는 메타 수준 제약의 분리와 규칙 로고를 통한 강제 아키텍처 무결성을 성공적으로 유지함을 나타낸다.

8.3 8.3 환각 감소

API 환각은 직접 프롬프트 기준선 대비 82% 감소했다. 실행자는 자신의 합성을 `SKILL.md` 에 문서화된 명시적 API 표면에 근거하도록 강제되었기 때문이다.

8.4 8.4 복합 효과

회고 지식의 축적은 측정 가능한 속도 향상으로 이어졌다. 반복 5회까지 중간 변환 시간은 28% 감소했고, 반복 10회까지는 35% 감소했다.

8.5 8.5 창발적 공학 기억

과거 수정에서 추출된 패턴이 자동으로 재사용되었으며, 마지막 세 번의 반복에서는 수동 개입이 단 한 번만 필요했던 반면, 처음 세 번의 반복에서는 네 번 필요했다.

9. 논의 (Discussion)

9.1 9.1 이것이 진정한 메타프로그래밍인가?

고전적 메타프로그래밍은 "프로그램을 조작하는 프로그램"으로 정의된다. 우리 시스템은 조작하는 프로그램을 LLM을 실행 엔진으로 사용하는 **준정형 아키텍처 메타 시스템**으로 대체한다. 그 조작은 더 이상 구문적 매크로 확장이 아니라, 제약 주도 변환이다. 우리는 이것이 새로운 형태의 메타프로그래밍—메타 언어가 형식적 프로그래밍 언어가 아니라 그림에도 실행 가능한, 인간이 작성 가능한 설계 언어인 메타프로그래밍—의 자격을 갖춘다고 주장한다. 모든 메타프로그래밍 시스템의 핵심 속성, 즉 메타 사양의 변경이 객체 프로그램의 예측 가능한 변경을 초래한다는 점은 완전히 충족된다.

9.2 9.2 한계

- **불완전한 형식 의미론:** LLM에 의한 자연어 규칙의 해석은 본질적으로 근사적이다. 우리 시스템이 비결정성을 극적으로 줄이지만, 완전히 제거하지는 못한다.
- **LLM 비결정성:** 출력의 작은 잔여 변동이 여전히 발생할 수 있다. 우리의 접근 방식은 많은 공학 작업에서 실질적으로 결정론적인 정도로 이를 줄이지만, 형식적 보장은 결정론적 실행 엔진을 필요로 할 것이다.
- **확장성:** 현재 시스템은 약 5,000줄의 코드베이스에서 테스트되었다. 더 큰 시스템으로 확장하려면 계층적 메타 문서와 더 정교한 캐싱이 필요할 수 있다.
- **의미론적 검증 비용:** 불변식 검증은 현재 수치 테스트 실행에 의존하며, 이는 대형 커널에 대해 비용이 많이 들 수 있다.

9.3 9.3 미래 연구

- **완전 형식적 메타 DSL:** Markdown에 내장될 수 있고 기호적 해석기에 의해 실행될 수 있는 작은 형식 정의의 제약 언어를 개발하여, LLM 해석에 대한 의존성을 줄인다.
- **그래프 기반 추론:** 평면적인 문서 그래프를 아키텍처 제약에 대한 더 강력한 추론을 가능하게 하는 지식 그래프로 교체한다.
- **분산 에이전트:** 설계자/실행자 모델을 각각 하위 시스템의 메타 문서를 소유하는 전문화된 에이전트 팀으로 확장한다.
- **런타임 적응:** 메타 문서를 하드웨어 텔레메트리에 기반하여 런타임에 매개변수를 조정하는 온라인 컨트롤러로 사용하는 방안을 탐구한다.

10. 결론 (Conclusion)

본 논문은 준정형 Markdown 설계 산출물을 실행 가능한 인과적 메타 표현으로 격상시키는 패러다임인 **에이전트 준비형 메타프로그래밍(Agent-Ready Metaprogramming)**을 제시했다. 전역 제약 추론을 지역적 코드 합성으로부터 아키텍처적으로 분리하고, 변환 파이프라인에 규칙 실행 추적과 메타 차이를 장착함으로써, 우리는 메타 문서의 수정이 생성 소프트웨어의 예측 가능하고 측정 가능한 변화를 초래함을 입증했다. GPU 커널 컴파일 사례 연구에서, 우리 시스템은 98%의 컴파일 성공률을 달성하고, 환각을 82% 감소시켰으며, 단순히 사람이 읽을 수 있는 파일을 편집하는 것만으로 하드웨어 독립적인 파라메트릭 생성을 가능하게 했다. 이 결과들은 준정형 설계 언어가 소프트웨어 변환을 위한 새로운 프론트엔드 역할을 하여, 인간의 설계 의도와 기계 실행 사이의 간극을 메울 수 있음을 입증한다. 우리는 이 연구가 아키텍처 문서가 수동적 설명이 아니라, 그것들이 기술하는 시스템을 형성하는 능동적이고 살아있는 제약이 되는 AI 네이티브 소프트웨어 공학으로의 길을 열 것이라고 믿는다.

— 끝 —